



North South University

Course name : CSE 225
Section : 12
Semester : Summer 24
HW 02 Topic : Arrays & Linked List
Date : 15-09-2024

Submitted To :

Dr. Md. Ahsan Habib (AHbb)

Submitted by :

NAME : Mohammad Tushar Abdullah

ID : 2323766042

HW - 02 (Arrays)

Question 01 : *What is the time complexity of removing an element from the end of an array and why?*

Ans : The time complexity of removing an element from the end of an array is $O(1)$ because we simply remove the last element but we don't need to shift other elements. The removal operation happens directly, so the time taken is constant,

Question 02 : *Consider the array [8, 3, 12]. What is the time complexity of adding the number 5 at the second position, and why?*

Ans : Given Array $A = [8, 3, 12]$. Time complexity of adding number 5 in the second position is $O(n)$. Because adding 5 in the middle of the array requires shifting the second half of the array to the right. Like adding to the beginning.

8	5	3	12
---	---	---	----

Question 03 : *If you want to remove the first element in the array [8, 3, 12], what would be the time complexity and the reason for this complexity?*

Ans : Given array, $A = [8, 3, 12]$. If I want to remove the first element that is 8 from the array, I need to shift other elements to the left to fill the gap. In that case time complexity will be $O(n)$.

3	12	
---	----	--

Question 04 : *What would be the time complexity of inserting an element at the third position of the array [8, 3, 12] if the array is not full?*

Ans : If we insert an element at the third position of the array $[8, 3, 12]$ we will need to shift other elements from third position to the right. The complexity of inserting an element at the 3rd position of the array is $O(n)$.

Because Time complexity of inserting an element at the 3rd position is $O(n-k)$ where k is the index position and here $k = 2$. But after skipping constant we can write $O(n)$.

Ans to the question no : 04

The time complexity of adding a new element at the beginning is $O(n)$. Because in the worst case let the array have n elements so to insert an element at the beginning we need to shift total n elements to one position right. Also the for loop runs n time.

So, The time complexity will be $O(n)$.

Ans to the question no : 05

The time complexity of removing an element from the end of an array is $O(1)$. Because, When we remove an element to the end of the array or at the end index of the array we don't need to shift the elements of the array to left or right. So we can see here the constant time complexity. Also we don't need to traverse all the elements of that array.

So, The time complexity will be $O(1)$ or constant time complexity.

Ans to the question no : 06

Given that,

array_addr = 1000

elem_size = 4

index = 5

first_index = 0

Now,

$$\text{offset} = \text{elem_size} * (\text{index} - \text{first_index})$$

$$\text{Or, offset} = 4 * (5-0)$$

$$\text{So, offset} = 20$$

Also,

$$\text{element_address} = \text{array_addr} + \text{offset}$$

$$\text{Or, element_address} = 1000 + 20$$

$$\text{So, element_address} = 1020$$

Thus, the address of the 5th element is 1020.

Accessing the 5th element takes constant time $O(1)$ because the calculation involves simple arithmetic (multiplication and addition). Since the array is stored in consecutive memory, the address of any element can be directly computed from its index, so the time taken doesn't depend on the array's size.

HW - 02 (LINKED LIST)

Question 01 : *What is the primary advantage of using a singly linked list over an array when inserting or deleting elements at arbitrary positions?*

Ans : while inserting and deleting elements in a linked list if we know the reference of that node where we need to do the operation we just need to update a few pointers but we don't need to shift elements. So insertion & deletion both operations can be done in constant time. Therefore time complexity $O(1)$.

When adding an element to any arbitrary position of an array needs to make space, that's why we need to shift other elements and that can not be done in constant time. Time complexity $O(n)$. In the same way if we delete an element from an array we need to fill the blank space with other elements and we need to shift elements that also can not be done in constant time. Time complexity $O(n)$.

Exception : While removing from an array if we are removing the last element or adding any element at the end position of the array in that case Time complexity $O(1)$.

Ans to the question no - 02

1. Why is the time complexity of the PushBack operation $O(n)$ in the absence of a tail pointer?

Ans : In PushBack operation we add elements at the back or end. If there is no tail then there is only one reference point and that is the Head in that case we have to start from the Head and traverse the full linked list. If there are a total of n elements we need to traverse all of them. That can not be done in constant time. So, Time complexity $O(n)$.

2. What modifications could be made to this operation to reduce its time complexity to $O(1)$?

Ans : to reduce the time complexity of this operation $O(1)$ we have to add a tail pointer to this linked list. As there is no tail pointer here so we need to traverse all elements of the list just because there is only one reference point and that is Head pointer. If we add a tail pointer that will be a new reference point and when we will be adding elements at the end we won't traverse the whole list again. That can be done in constant time & reduce time complexity. So, Time complexity $O(1)$.

3. Discuss the trade-offs between maintaining a tail pointer and the overhead it introduces in the linked list.

Ans :

- **Efficient PushBack operation :** Adding elements to the end of the list becomes an $O(1)$ operation, as the tail pointer provides direct access to the last node.
- **Increased Memory Usage :** The tail pointer adds extra memory overhead by storing an additional reference alongside the head pointer.
- **Additional Complexity :** Managing the tail pointer requires extra care during modifications to the list. For instance, when nodes are inserted or deleted, the tail pointer must be updated accordingly, adding complexity to the list operations.

Ans to the question no - 03

To insert a new node at the front of a singly linked list, we need to simply create the new node and adjust its next pointer to point to the current head node. Then we need to update the head pointer to refer to this new node.

Since both operations are performed in constant time, the overall time complexity is $O(1)$.

On the other hand, Adding an element to the front of an array requires shifting all existing elements one position to the right to make space for the new element. This can't be done in constant time. Time complexity $O(n)$.

Therefore, The PushFront operation in a singly linked list is more efficient than adding an element to the front of an array.

Ans to the question no - 04

1. In a singly linked list, even though we have a tail pointer that allows direct access to the last node, we cannot directly access the second-to-last node. This is because each node only has a pointer to the next node, and there is no way to traverse backward. To remove the last node (PopBack), we need to traverse the entire list starting from the head node.

Since this requires visiting all n nodes in the worst case, So Time complexity is $O(n)$.

2. By keeping track of both the last node and the second-to-last node, we could do the PopBack operation in constant time. This is because we wouldn't need to go through the whole list to find the second-to-last node. we would need to update the second-to-last pointer every time we add or remove a node near the end of the list.

In that case time complexity is $O(1)$.

3. In a doubly linked list, each node has two pointers: one pointing to the next node and one pointing to the previous node. This allows us to directly access both the last node and the second-to-last node.

For the PopBack operation in a doubly linked list, we can access the last node directly using the tail pointer in constant time, $O(1)$. From there, we can use the prev pointer of the last node to instantly access the second-to-last node, also in $O(1)$. Once we have the second-to-last node, we update the tail pointer to point to it and then remove the last node. This makes the entire PopBack operation efficient, with a time complexity of $O(1)$.

Because we can access the second-to-last node without having to traverse the list, the time complexity of the PopBack operation in a doubly linked list becomes $O(1)$.

Ans to the question no - 05

1. Why is the time complexity of the PopBack operation in a doubly linked list $O(1)$?

Ans : In a doubly linked list, the PopBack operation has a time complexity of $O(1)$ because we can directly access both the last node and the second-to-last node using the pointers in the list. The tail pointer gives us direct access to the last node in constant time, and the prev pointer of the last node helps us reach the second-to-last node without traversing the entire list. This allows us to update the tail pointer and remove the last node quickly, making the overall time complexity $O(1)$.

2. How do the prev and next pointers enhance the efficiency of this operation compared to a singly linked list?

Ans : In a singly linked list, each node only has a pointer to the next node, so when we do the PopBack operation, we have to traverse the entire list starting from the head to reach the second-to-last node. This takes $O(n)$ time since we need to go through each node one by one.

But in a doubly linked list, every node has two pointers: *one to the next node* and *one to the previous node*. These next and prev pointers let us directly access the last node using the tail pointer, and the prev pointer of the tail helps us get to the second-to-last node without having to traverse the list. So, the PopBack operation becomes much faster and takes constant time, $O(1)$.

3. *What are the memory trade-offs between maintaining a doubly linked list and a singly linked list?*

Ans : While a doubly linked list makes operations like PopBack faster, it uses more memory than a singly linked list. In a singly linked list, each node only has one pointer (to the next node), but in a doubly linked list, each node has two pointers: one for the next node and one for the previous node. So, a doubly linked list takes up more memory per node.

Also, the extra pointers make operations like inserting and deleting more complicated, since we need to update both the next and prev pointers. So, even though a doubly linked list is faster for some operations, it uses more memory and is harder to manage than a singly linked list.

